



Universidad de Concepción

Facultad de Ingeniería

Departamento de Informática y Ciencias de la Computación

Asignatura: Inteligencia Artificial (501351)

Profesor: Julio Godoy

Tarea 1: Escape de la Torre

Autor: Francisco Arentsen San Martín

Fecha: Septiembre de 2026

Índice

1. Introducción	2
1.1. Contexto y tema del informe	2
1.2. Objetivos	2
1.3. Metodología de desarrollo y uso de IA generativa	2
2. Estructura del Proyecto	3
2.1. Componentes	3
2.2. Flujo de un turno	3
2.3. Interfaz común de los gestores	4
2.4. Métricas	4
3. Ambiente	4
3.1. Origen de los mapas	4
3.2. Procesamiento	5
3.3. La clase Ambiente	5
4. Agentes	5
4.1. Enfoque general	5
4.2. BFS	5
4.3. DFS	5
4.4. GBFS (Greedy Best-First Search)	5
4.5. A*	6
4.6. Algoritmo Genético (GA)	6
4.7. Problemas encontrados durante el desarrollo	6
5. Experimentos	6
5.1. Descripción de las pruebas	6
5.2. Resultados	6
5.3. Discusión de los resultados	6
6. Conclusiones	8
7. Cosas a Mejorar	8
8. Links y Referencias	8

1. Introducción

1.1. Contexto y tema del informe

Este informe corresponde a la Tarea 1 “Escape de la Torre”. El enunciado plantea un edificio en llamas donde un grupo de personas debe alcanzar la única zona de escape de cada piso antes de que el fuego las alcance. Los pasillos son estrechos, por lo que una alta concentración de personas en una misma vía genera embotellamientos que retrasan el flujo.

El informe describe el diseño, la implementación y la evaluación de un simulador de evacuación en una grilla, donde:

- Los agentes se mueven de forma discreta (arriba, abajo, izquierda, derecha o esperar).
- El fuego se propaga cada k turnos hacia casillas adyacentes, de forma irreversible; las casillas quemadas son intransitables y los agentes alcanzados por el fuego mueren.
- Dos agentes no pueden ocupar la misma casilla, y las zonas con tránsito reciente elevado tienen un costo mayor (congestión).

Sobre este ambiente se implementaron cinco estrategias, una por cada paradigma pedido:

- **Búsqueda no informada** (2 algoritmos): BFS y DFS.
- **Búsqueda informada** (2 algoritmos, con heurística admisible): GBFS y A*, ambos con distancia de Manhattan.
- **Optimización bioinspirada** (1 algoritmo): Algoritmo Genético (GA).

1.2. Objetivos

Objetivo general: Implementar el sistema de navegación y toma de decisiones de los agentes bajo tres paradigmas (búsqueda no informada, informada y bioinspirada), minimizando las pérdidas humanas y mitigando la congestión, y compararlos mediante un benchmark controlado.

Objetivos específicos:

- Diseñar un ambiente por capas (estructura, fuego, agentes) reutilizable por todos los algoritmos.
- Implementar los cinco agentes con una interfaz común, de modo que sean intercambiables sin modificar el resto del sistema.
- Definir una función de costo por congestión (lineal, cuadrática o exponencial) e incorporarla en los algoritmos ponderados.
- Evaluar cada combinación (mapa, algoritmo) con las métricas obligatorias del enunciado: tasa de supervivencia y estadísticos descriptivos (media, desviación estándar, mínimo, máximo) de los turnos de despeje.
- Analizar ventajas, limitaciones y costos de cada enfoque.

1.3. Metodología de desarrollo y uso de IA generativa

El trabajo se desarrolló en las siguientes etapas:

1. Definición de la idea: reglas del ambiente, entidades y métricas.
2. Escritura del pseudocódigo de las clases (**Ambiente**, **Controlador**, y **Agente** y **Gestor** de cada algoritmo).
3. Entrega del pseudocódigo a una IA generativa para que generara las clases en Python.
4. Pruebas sobre el código generado y refactorización a partir de los errores y comportamientos indeseados encontrados (incluyendo varias rondas de depuración guiada de bugs de concurrencia entre agentes, descritas en la sección 4.7).
5. Ejecución del benchmark y análisis de resultados.

De acuerdo con lo que exige el enunciado, se declara explícitamente que:

- El código de los agentes, el ambiente, el controlador y el benchmark fue generado con IA generativa (Claude, de Anthropic) a partir de pseudocódigo propio, y luego probado, depurado y refactorizado por el autor a lo largo de varias iteraciones.
- El algoritmo genético se adaptó a partir del repositorio **Genetic-Algorithm-Pathfinding** (ver sección 8).
- Los mapas provienen de **Moving AI Lab** (ver sección 3), editados y convertidos al formato del proyecto con asistencia de la misma herramienta.

2. Estructura del Proyecto

2.1. Componentes

El proyecto se divide en tres grupos de módulos con responsabilidades separadas:

(a) **Ambiente (`src/core/ambiente.py`):** Representa el mundo: el mapa, el fuego, la posición de los agentes y el contador de tránsito para la congestión. No toma decisiones; solo almacena estado y ofrece operaciones sobre él (propagar fuego, colocar agentes, calcular costo de una casilla).

(b) **Algoritmos (`src/algorithms/`: `bfs.py`, `dfs.py`, `gbfs.py`, `a_star.py`, `genetic.py`):** Cada archivo define dos clases: un **Agente** (toma las decisiones individuales de movimiento) y un **Gestor** (coordina a todos los agentes de esa estrategia). Todos los gestores exponen la misma interfaz, lo que permite que el controlador y el benchmark los traten de forma idéntica.

(c) **Controlador (`src/core/controlador.py`):** Clase **ControladorSimulacion**. Orquesta el ciclo de vida de un turno. Es la interfaz única que usan tanto la GUI como el benchmark, de modo que ambos ejecutan exactamente la misma lógica.

Adicionalmente, `benchmark.py` (`src/simulation/`) ejecuta los experimentos masivos y guarda los resultados en un CSV, y `gui.py` (`src/ui/`) permite visualizar la simulación en tiempo real.

2.2. Flujo de un turno

Cada llamada a `ControladorSimulacion.avanzar_turno()` ejecuta, en orden:

1. **Movimiento:** `gestor.mover_turno()`. Cada agente activo se mueve una casilla (o espera) en secuencia, y la Capa 2 se resincroniza después de cada agente para evitar colisiones dentro del mismo turno.
2. **Registro de tránsito:** por cada agente vivo y no escapado se incrementa el contador de tránsito de su casilla actual; luego todo el contador decae con un factor de 0.7. Así la congestión refleja el tráfico reciente y no se acumula indefinidamente.
3. **Propagación del fuego:** cada k_{fuego} turnos el fuego se expande una casilla en las cuatro direcciones.
4. **Impacto del fuego:** `gestor.evaluar_impacto_fuego()` marca como muertos los agentes cuya casilla quedó en llamas.
5. **Registro de escapes:** se anota en qué turno escapó cada agente (solo la primera vez).

La simulación termina cuando todos los agentes escaparon o murieron, o cuando se alcanza el límite `max_turnos` (1000 por defecto).

2.3. Interfaz común de los gestores

Todos los gestores implementan: `mover_turno()`, `evaluar_impacto_fuego()`, `simulacion_terminada()` y `_sincronizar_capa_agentes()`. Todos los agentes tienen los atributos `id`, `pos_actual`, `vivo` y `escapado`. Los movimientos son de 4-conectividad (arriba, abajo, izquierda, derecha).

2.4. Métricas

`obtener_estadisticas()` entrega el número de sobrevivientes, el total de agentes y los turnos de despeje. Los turnos de despeje se definen como el turno en que escapó el último sobreviviente, y no el turno en que terminó la simulación (ambos pueden diferir si el último evento fue la muerte de un agente rezagado). Si en una iteración no escapa ningún agente, esa iteración no aporta a las estadísticas de turnos (solo a la tasa de supervivencia).

3. Ambiente

3.1. Origen de los mapas

Los mapas se obtuvieron de Moving AI Lab (<https://www.movingai.com/>), un repositorio de mapas de referencia para problemas de pathfinding. Se usaron tres mapas de 64x64 celdas para representar los niveles de dificultad topológica del enunciado:

- **Mapa 1** (`ambiente1.txt`): alta densidad de obstáculos / cuello de botella (pasillos angostos que convergen hacia la salida).
- **Mapa 2** (`ambiente2.txt`): densidad media / laberinto corporativo (salas conectadas por intersecciones y cruces ciegos).

- **Mapa 3** (`ambiente3.txt`): baja densidad / dispersión abierta (semiabierto, con múltiples rutas alternativas).

Particularidades encontradas al revisar los tres mapas en detalle:

- El ancho de la salida varía: Mapa 1 tiene 4 celdas de ancho, Mapa 2 tiene 2 celdas, y Mapa 3 tiene una sola celda.
- El Mapa 2 define dos orígenes de fuego en el archivo (dos celdas con el dígito 3), pero `Ambiente._procesar_capas()` solo toma el primero que encuentra, dejando el segundo inactivo.
- Estimación de márgenes temporales antes de que el fuego alcance la salida ($k_{\text{fuego}} = 3$):
 - **Mapa 1:** partida $\rightarrow$ salida = 117 pasos; fuego $\rightarrow$ salida = 66 celdas $\rightarrow$ fuego llega turno ~ 198 ; **margen ~ 81 turnos.**
 - **Mapa 2:** partida $\rightarrow$ salida = 119 pasos; fuego $\rightarrow$ salida = 61 celdas $\rightarrow$ fuego llega turno ~ 183 ; **margen ~ 64 turnos.**
 - **Mapa 3:** partida $\rightarrow$ salida = 150 pasos; fuego $\rightarrow$ salida = 59 celdas $\rightarrow$ fuego llega turno ~ 177 ; **margen ~ 27 turnos.**

3.2. Procesamiento

Los mapas originales fueron editados a dimensiones 64x64 y convertidos al formato numérico mediante `src/core/conversor_de_mapas.py`:

- 0: espacio libre, 1: pared, 2: salida, 3: origen del fuego, 4: punto de partida.

3.3. La clase Ambiente

El mapa se carga en una matriz 3D de NumPy de tamaño (64, 64, 3):

- **Capa 0 (estructural):** 0 libre, 1 pared, 2 salida.
- **Capa 1 (fuego):** 1 si está en llamas.
- **Capa 2 (agentes):** 1 si hay un agente vivo y activo.

Función de costo por congestión: Calcula el costo de transitar por la casilla (f, c) según su ocupación de tránsito reciente $O_{f,c}$:

- **Lineal:** $C(f, c) = 1 + \alpha \cdot O_{f,c}$
- **Cuadrático (defecto):** $C(f, c) = 1 + \alpha \cdot O_{f,c}^2$
- **Exponencial:** $C(f, c) = \exp(\alpha \cdot O_{f,c})$

El costo siempre es ≥ 1 , asegurando la admisibilidad de la heurística de Manhattan en A*.

4. Agentes

4.1. Enfoque general

Los cinco algoritmos comparten la misma arquitectura base:

- **Agente:** almacena su estado interno, ruta y posición actual.
- **Gestor:** coordina el ciclo de turnos, sincroniza la Capa 2 y evalúa el impacto del fuego.

4.2. BFS

Explora con una cola FIFO. Cómputo de ruta más corta en número de pasos (costo uniforme = 1). Replanifica si la siguiente casilla de la ruta está bloqueada por otro agente o fuego.

4.3. DFS

Evaluación de vecinos locales. Distingue entre *bloqueo permanente* (muro, fuego, visitado) y *bloqueo temporal* (agente). Ante un bloqueo temporal, el agente espera en el lugar en vez de retroceder. Presenta riesgo de *deadlock* estructural en pasillos angostos.

4.4. GBFS (Greedy Best-First Search)

Cola de prioridad ordenada por $(h, C_{\text{congestión}}, \text{posición})$, donde h es la distancia de Manhattan a la salida. No acumula el costo g . Voraz y rápido.

4.5. A*

Cola de prioridad según $f = g + h$, donde g acumula el costo de tránsito con congestión $C(f, c) \geq 1$ y h es Manhattan. Incluye descartes de nodos obsoletos y replanificación periódica cada 4 pasos.

4.6. Algoritmo Genético (GA)

Adaptado de Genetic-Algorithm-Pathfinding.

- **Cromosoma:** secuencia de 20 movimientos.
- **Población y Evolución:** 25 individuos, 15 generaciones, cruce en un punto ($p = 0,8$), mutación ($p = 0,05$), selección por torneo ($k = 3$).
- **Fitness:** premia acercarse a la salida ($1000 + (\text{longitud} - \text{pasos}) \cdot 10$ si llega), penaliza casillas repetidas (0,5) y congestión (0,3).

4.7. Problemas encontrados durante el desarrollo

- Resincronización inmediata de la Capa 2 tras cada movimiento individual para evitar colisiones intra-turno.
- Ajuste de métricas para registrar el turno de escape exacto de cada agente individual.
- Corrección del GA al cambiar la selección por ruleta (sensible a fitness negativo) por selección por torneo.

5. Experimentos

5.1. Descripción de las pruebas

Se realizaron 128 iteraciones independientes por combinación de (Mapa, Algoritmo), evaluando un total de 3200 agentes por combinación (25 agentes por iteración). Se utilizaron semillas fijas $((i + 1) \cdot 100)$ para asegurar reproducibilidad exacta.

5.2. Resultados

Tabla 1: Tasa de supervivencia y turnos de despeje (128 iteraciones por combinación).

Mapa	Algoritmo	Superv. (%)	Media	Desv. Est.	Mín	Máx
Mapa 1	BFS	100.00	126.93	1.22	124	130
Mapa 1	DFS	0.00	N/A	N/A	N/A	N/A
Mapa 1	GBFS	100.00	135.80	2.61	129	142
Mapa 1	A*	100.00	128.95	2.24	123	134
Mapa 1	GA	4.50	154.77	15.37	117	186
Mapa 2	BFS	37.50	120.08	14.46	106	161
Mapa 2	DFS	0.00	N/A	N/A	N/A	N/A
Mapa 2	GBFS	49.75	152.66	3.51	144	167
Mapa 2	A*	39.38	157.41	3.51	152	163
Mapa 2	GA	0.00	N/A	N/A	N/A	N/A
Mapa 3	BFS	97.97	160.19	3.32	152	164
Mapa 3	DFS	0.00	N/A	N/A	N/A	N/A
Mapa 3	GBFS	45.88	161.62	2.52	151	164
Mapa 3	A*	75.88	161.70	3.44	142	164
Mapa 3	GA	0.00	N/A	N/A	N/A	N/A

Tabla 2: Tiempo de ejecución total por combinación (segundos).

Algoritmo	Mapa 1 (s)	Mapa 2 (s)	Mapa 3 (s)
BFS	460.98	261.21	95.09
DFS	654.44	84.28	184.95
GBFS	81.70	76.43	103.45
A*	1223.08	614.02	736.14
GA	1541.58	1986.95	2972.86

5.3. Discusión de los resultados

- **Mapa 1:** BFS, GBFS y A* alcanzaron el 100% de supervivencia debido al amplio margen temporal (~ 81 turnos) y al ancho de la salida (4 celdas).
- **Mapa 2:** Muestra la mayor dificultad para los algoritmos clásicos (supervivencia entre 37.5% y 49.75%) debido a la complejidad del laberinto y el menor ancho de salida (2 celdas).

- **Mapa 3:** Muestra una contradicción con la hipótesis de dificultad topológica: aunque no tiene muros internos densos, la salida tiene solo 1 celda de ancho y el margen temporal es el más reducido (~ 27 turnos). BFS logró 97.97 % al no desviarse, mientras que A* (75.88 %) sufrió por rodeos causados por la función de congestión.
- **DFS y GA:** DFS obtuvo 0 % de supervivencia en todos los casos debido a *deadlocks* por bloqueo mutuo. El GA resultó computacionalmente ineficiente y con un desempeño casi nulo (4.5 % en Mapa 1 y 0 % en el resto).

6. Conclusiones

- Se construyó un ambiente multicapa y un controlador único reproducibles con semillas por iteración.
- BFS resultó ser la estrategia más robusta en general, demostrando que la mayor complejidad algorítmica no garantiza mayor supervivencia si la función de costo no contempla explícitamente el avance del fuego.
- La dificultad real de un escenario depende principalmente de la relación entre el ancho del cuello de botella de escape y el tiempo crítico de llegada del fuego.

7. Cosas a Mejorar

- Resolver el *deadlock* de DFS mediante un mecanismo de cesión de paso tras N turnos de espera consecutiva.
- Replantear la función de fitness del GA o la frecuencia de re-evolución para reducir su costo computacional.
- Incorporar el riesgo de propagación del fuego en la función de costo de A* y GBFS.
- Corregir la activación del segundo foco de fuego en el Mapa 2.
- Paralelizar la ejecución del benchmark.

8. Links y Referencias

- **Repositorio de origen del Algoritmo Genético:**
<https://github.com/Pojzo/Genetic-Algorithm-Pathfinding.git>
- **Moving AI Lab (Repositorio de mapas):**
<https://www.movingai.com/>
- **Repositorio del proyecto:**
https://github.com/Farentsen999/Tarea1_IA.git
- **Enunciado:** Tarea 1 “Escape de la Torre”, asignatura Inteligencia Artificial, Universidad de Concepción, Septiembre de 2026.